

Tarsau – ASCII Tabanlı Arşivleme Sistemi

SİSTEM PROGRAMLAMA DERSİ (2025-2026 BAHAR DÖNEMİ)

Teknik Tasarım ve Akademik Analiz Raporu

Hazırlayanlar: Fatih Kaya - G231210072

Bölüm: Bilgisayar Mühendisliği

Üniversite: Sakarya Üniversitesi

Tarih: 22 Mayıs 2026

GitHub:

 [GitHub - katze2023/Tarsau: Sakarya Üniversitesi Sistem Progr...](https://github.com/katze2023/Tarsau)

1. Projenin Amacı

1.1 Projenin Genel Amacı

Bu projenin temel amacı, Linux/Unix ortamında standart bir arşivleme aracı (tar, zip, rar) mantığıyla çalışan; ancak hiçbir sıkıştırma algoritması içermeyen “tarsau” adlı bir komut satırı programı geliştirmektir. Program, birden fazla ASCII metin dosyasını alarak bunları tek bir “.sau” uzantılı arşiv dosyasında birleştirmekte, gerektiğinde bu arşiv dosyasını çözümleyerek dosyaları orijinal hâllerinde (izinler dahil) hedef bir dizine geri açabilmektedir.

Proje, ders kapsamında sistem programlama disiplininin temel kavramlarının uygulamalı olarak öğrenilmesini hedeflemektedir. Bu kavramlar arasında dosya tanımlayıcıları (file descriptors), POSIX sistem çağrıları, byte-seviyesinde veri akışı yönetimi, dosya meta verileri (metadata), izin bitleri (mode bits) ve izin işlemleri yer almaktadır.

1.2 Programın Komut Satırı Sentaksı

Arşivleme komutu : -b

Bu komut, listelenen giriş dosyalarını birleştirip s1.sau adlı arşiv dosyasını üretir ve başarılı olduğunda Dosyalar birleştirildi. mesajı verir.

Çıkarma komutu : -a

Bu komut ise s1.sau arşivini çözümleyip d1 dizinine açar; sonunda d1 dizininde t1, t2, t3, t4.txt ve t5.dat dosyaları açıldı. mesajını yazdırır.

1.3 Proje Belgesinde Belirtilen Kısıtlamalar

Program tasarlanırken proje belgesinde belirtilen aşağıdaki kısıtlamaların tamamına uyulması zorunlu kılınmıştır:

Giriş dosyası kısıtlamaları:

1. Giriş dosyaları yalnızca **ASCII** karakter setine sahip metin dosyaları olabilmektedir (karakter başına 1 byte). Binary (ikili) içerikli dosyalar kabul edilmemektedir.
2. Giriş dosyalarının toplam boyutu **200 MB**'ı geçemez.
3. Aynı anda arşivlenebilecek dosya sayısı en fazla **32** olabilir.

Arşiv adı kuralı:

1. -o parametresi verilmediği durumda arşiv dosyası adı varsayılan olarak a.sau olmalıdır.

Çıkarma modu kuralları:

- -a parametresinden sonra en fazla iki argüman alınmalıdır: arşiv dosyası adı ve hedef dizin adı.
- Arşiv dosyasının uzantısı .sau olmak zorundadır; aksi takdirde Arşiv dosyası uygunsuz veya bozuk! mesajı verilmelidir.
- Hedef dizin parametresi verilmediğinde dosyalar geçerli (çalışılan) dizine açılmalıdır.
- Belirtilen hedef dizin mevcut değilse program tarafından oluşturulmalıdır.
- Çıkarılan dosyaların, arşivlendikleri anki ile **aynı izin bitlerine** (okuma/yazma/çalıştırma) sahip olması gerekmektedir.

Hata yönetimi:

- ASCII uyumsuz bir dosya verildiğinde program çökmeden, <dosya_adı> giriş dosyasının formatı uyumsuzdur! mesajıyla ve sorunsuz bir şekilde sonlanmalıdır.
- Program hiçbir koşulda ani çökme (crash) yaşamamalı; tüm hatalı durumlar kontrollü biçimde ele alınmalıdır.

1.4 Sistem Çağrısı Kısıtı

Projenin en belirleyici teknik kısıtı, dosya işlemleri için yalnızca POSIX sistem çağrılarının (open, read, write, close, lseek, stat, fstat, mkdir, chmod) kullanılmasıdır. stdio.h kütüphanesindeki fopen, fread, fwrite, fprintf, printf, fscanf, sscanf gibi yüksek seviyeli tamponlu I/O fonksiyonlarının kullanımı **kesinlikle yasaklanmıştır**.

Bu kısıt, ilk bakışta yalnızca bir API tercihi gibi görünse de aslında programcıyı çok daha temel sorunlarla yüzleştirmektedir:

1. “printf” ve “fprintf” gibi fonksiyonlar veriyi doğrudan kernel'e iletmek yerine önce kullanıcı uzayındaki (user-space) bir tampona yığar. Bu durum, program beklenmedik şekilde sonlanırsa mesajların hiç yazılmamasına yol açabilir.
2. write() sistem çağrısının her zaman istenen byte sayısını tek seferde yazmayı garanti etmemesi (*partial write*), büyük dosyalar üzerinde çalışırken arşivin sessizce bozulmasına neden olabilir. Bu durum elle, döngülerle yönetilmek zorundadır.
3. read() çağrısının da benzer şekilde *partial read* davranışı göstermesi

mümkündür ve aynı dikkat gerekir.

4. Tampon yönetimi, dosya konumu (lseek) takibi ve veri akışının doğru kopyalanması doğrudan programcının sorumluluğundadır.

Bu sebeplerle projenin önemli bir kısmı; partial I/O yönetimi, UTF-8 uyumlu mesaj çıktıları, bellek tahsisi (malloc) ve serbest bırakma (free) kontrolleri gibi düşük seviyeli teknik problemlerin çözümüne ayrılmıştır.

1.5 Geliştirme Süreci

Proje C11 standardı kullanılarak GNU Compiler Collection (gcc) ile derlenmiştir. Derleme otomasyonu için Makefile yapılandırılmış; standart make, hata ayıklama modunu destekleyen make debug ve en yüksek seviye optimizasyon için make release hedefleri sağlanmıştır. Geliştirme ve test ortamı olarak Windows üzerinde WSL (Windows Subsystem for Linux) ile çalışan Ubuntu çekirdeği ve Visual Studio Code editörü kullanılmıştır. Ek doğrulama için doğrudan Linux dağıtımı kullanan ayrı bir bilgisayarda da testler tekrarlanmıştır. Versiyon kontrolü Git ile yapılmış, kaynak kodlar GitHub'da yayımlanmıştır.

2. Kullanılan Teknolojiler

Proje geliştirme sürecinde aşağıdaki teknolojiler ve standartlar kullanılmıştır:

1. Programlama Dili: C (C11 Standardı)
2. Sistem Çağrıları (POSIX API): open(), read(), write(), close(), lseek(), stat(), fstat(), mkdir(), chmod()
3. Derleyici: GNU Compiler Collection (gcc)
4. Otomasyon: Makefile (Gelişmiş build, debug ve release modları)
5. Versiyon Kontrol: Git & GitHub
6. Geliştirme Ortamı: Linux / WSL (Windows Subsystem for Linux), VS Code

3. Sistem Mimarisi ve Bellek Yönetimi

3.1 Modüler Dosya Yapısı

Proje, tek bir büyük .c dosyası yerine birbirinden bağımsız beş dosyaya bölünmüştür. Bu tercih, her dosyanın yalnızca kendi sorumluluk alanındaki işleri yapması ve modüller arasındaki bağımlılıkların kontrol altında tutulması amacıyla alınmıştır. “sau_common.h” dışında hiçbir modül diğerinin dahili yapısını doğrudan kullanmamaktadır.

Dosya	Sorumluluk	Bağımlılık
sau_main.c	CLI (Command Line Interface) argümanlarını ayrıştırır, sınır kontrollerini yapar ve iş akışını ilgili modüllere (arşivleme veya çıkarma) yönlendirir.	sau_common.h
sau_archive.c	Arşiv oluşturma motorudur. Dosyaların boyut ve izin metadatalarını toplar, .sau formatına uygun başlık (header) kısmını oluşturur ve verileri hedefe yazar.	sau_common.h, sau_utils.c
sau_extract.c	Arşiv açma motorudur. Metadatayı analiz eder (parsing), gerekli hedef dizinleri oluşturur ve baytları ayrıştırarak dosyaları orijinal izinleriyle diske yazar.	sau_common.h, sau_utils.c
sau_utils.c	Sistem geneli yardımcı fonksiyonları barındırır: ASCII doğrulama, güvenli byte aktarımı (push/pull) ve dosya izinlerinin okunması işlemleri buradadır.	sau_common.h
sau_common.h	Sistem genelinde paylaşılan sabitleri (200 MB ve 32 dosya limiti dahil),	—

--	--	--

Bu yapının derleme sürecine yansıyan somut avantajı şudur: yalnızca “sau_extract.c” değiştirildiğinde Makefile, bağımlılık kuralları sayesinde diğer dosyaları yeniden derlemek yerine sadece “sau_extract.o” u günceller. Bu durum özellikle büyük projelerde derleme süresini ciddi ölçüde kısaltır.

3.2 Bellek Yönetimi Yaklaşımı

Projede bellek yönetimi konusunda iki temel ilke benimsenmiştir: mümkün olduğunca yığın (stack) kullan, gerektiğinde heap'ten ayır ama her serbest bırakmayı garanti et.

Stack tabanlı tamponlar: Byte aktarımı için kullanılan sau_push_bytes fonksiyonundaki 64 KB'lık buf[SAU_IO_BUFFER] tamponu yığın üzerinde tanımlanmıştır. Bu sayede her aktarım için ayrı bir malloc/free çağrısına gerek kalmamakta, tahsis hatası riski tamamen ortadan kalkmaktadır. Benzer şekilde metadata derlemesinde kullanılan metadata_body[SAU_META_BUF_SIZE] (10.240 byte) yığında bildirilmiştir.

Heap tabanlı tahsisler: Arşiv açma sırasında metadata bölümünün boyutu önceden bilinmediğinden, bu veriyi okumak için malloc(meta_size + 1) çağrısıyla heap'ten dinamik tahsis yapılmaktadır. Bu tahsis için iki seviyeli güvence eklenmiştir:

1. malloc çağrısının dönüş değeri kontrol edilmekte; NULL dönmesi durumunda program kontrollü şekilde sonlandırılmaktadır.
2. Fonksiyon içindeki her hata dalı (comma1 == NULL, read hatası, geçersiz format vb.) free(meta_body); close(fd); exit(1) üçlüsünü çalıştırmaktadır. Böylece hiçbir kod yolunda tahsis edilen belleğin serbest bırakılmaması (memory leak) durumu oluşmaz.

Sabit boyutlu meta veri yapısı: Her giriş dosyası için meta veriler SauEntry yapısında tutulmaktadır:

```

34  /* ===== Dosya Girdi Yapısı ===== */
35  typedef struct
36  {
37      char    filename[256];
38      mode_t  permissions;
39      size_t  size;
40  } SauEntry;
41

```

Maksimum 32 dosya kabul edildiğinden, entries[SAU_MAX_FILES] dizisi de stack üzerinde sabit boyutlu olarak tanımlanmıştır. Bu yaklaşım, runtime'da bellek tükenmesi (out of memory) ihtimalini sıfırlamaktadır.

3.3. ".sau" Dosya Formatı

“.sau” dosya formatı tasarlanırken temel öncelik sadelik ve ayrıştırma kolaylığı olmuştur. Dosya iki ana bölümden oluşmakta, bölümler arasında herhangi bir boşluk ya da ayırıcı karakter bulunmamaktadır:

Bölüm	Boyut	Açıklama	Örnek İçerik
Header Size Field	10 Byte	Metadata bölümünün bayt cinsinden uzunluğunu tutan, soldan sıfır dolgulu onluk tabanda ASCII sayı.	0000000042
Metadata Body	Değişken	Her dosya için ayrılmış bilgileri içerir. Format: `	dosya_adi,izin,boyut
Raw File Data	Değişken	Arşivlenen dosyaların ham metin içerikleri, aralarında hiçbir ayırıcı bayt olmadan ardışık olarak tutulur.	Merhaba Dunyals -I calisti

4. TEMEL FONKSİYONLARIN AÇIKLANMASI

Proje mimarisi modüler bir yapıda tasarlanmış olup, sistemin çekirdeğini oluşturan temel fonksiyonların işleyişi ve görevleri aşağıda özetlenmiştir:

main() (sau_main.c) Programın giriş noktasıdır ve komut satırı argümanlarının (CLI) yönetimini üstlenir. Kullanıcıdan gelen -a (çıkarma) veya -b (arşivleme) bayraklarını denetler, dosya limiti ve uzantı (.sau) gibi sınır kontrollerini gerçekleştirir. Hatalı parametre girişlerini engelledikten sonra, iş akışını ilgili ana motorlara (sau_build_archive veya sau_extract_archive) yönlendirir.

```
54 /*
55  * sau_push_bytes - Kaynak dosyadan hedef dosyaya veri kopyalar (Arşivleme için)
56  *
57  * İşleyiş:
58  * - src_fd (kaynak dosya) üzerinden n byte okur ve dst_fd (hedef dosya) üzerine yazar.
59  * - read ve write sistem çağrılarının tam olarak istenen miktarda byte
60  *   işlememesi (partial read/write) riskine karşı döngülerle korunmuştur.
61  */
62 ssize_t sau_push_bytes(int src_fd, int dst_fd, size_t n)
63 {
64     unsigned char buf[SAU_IO_BUFFER];
65     size_t total_written = 0;
66
67     /* Kopyalanması gereken toplam byte sayısına ulaşana kadar döngüye girer */
68     while (total_written < n)
69     {
70         /* Okunacak byte miktarı tampon boyutundan büyükse parça parça okunur */
71         size_t to_read = (n - total_written > SAU_IO_BUFFER)
72             ? SAU_IO_BUFFER
73             : (n - total_written);
74         ssize_t r = read(src_fd, buf, to_read);
75         if (r <= 0) return -1; /* Okuma hatası veya beklenmeyen dosya sonu (EOF) */
76
77         /* Okunan veriyi hedef dosyaya yazar */
78         ssize_t written = 0;
79         while (written < r)
80         {
81             ssize_t w = write(dst_fd, buf + written, (size_t)(r - written));
82             if (w <= 0) return -1; /* Yazma hatası */
83             written += w;
84         }
85         total_written += (size_t)r;
86     }
87     return (ssize_t)total_written;
88 }
89
```

sau_build_archive() (sau_archive.c) Arşiv oluşturma sürecini yöneten ana motordur. Kendisine iletilen girdi dosyalarını sırasıyla tarar, boyutlarını ve izinlerini (stat yardımıyla) elde eder. Elde edilen bu bilgileri [dosya_adi,izin,boyut] formatında birleştirerek bellekte bir üstveri (metadata) bloğu oluşturur. Arşiv dosyasını oluşturduktan sonra en başa 10 baytlık metadata uzunluğunu, ardından metadata bloğunu ve son olarak dosyaların ham (raw) içeriklerini hiçbir ayırıcı kullanmadan sırayla yazar.

```
49 /* ---- Doğrulama ve metadata derleme ---- */
50 for (int i = 0; i < file_count; i++) {
51     char *path = argv[i + 1];
52
53     /* -----
54     * Dosya adında geçersiz karakter kontrolü
55     * '|' -> metadata ayırıcısı ile çakışır
56     * ',' -> alan ayırıcısı ile çakışır
57     * ----- */
58     if (strchr(path, '|') || strchr(path, ',')) {
59         SAU_ERR("Geçersiz dosya adı karakteri.\n");
60         exit(1);
61     }
62
63     int fd = open(path, O_RDONLY);
64     if (fd == -1) { perror(path); exit(1); }
65
66     /* ASCII doğrulama - geçersiz mesaj basıp exit(0) yapar */
67     sau_validate_ascii_stream(fd, path);
68
69     /* stat() ile boyut ve izin bilgisi */
70     struct stat st;
71     if (fstat(fd, &st) == -1) { perror(path); exit(1); }
72
73     entries[i].size = (size_t)st.st_size;
74
75     total_size += entries[i].size;
76     if (total_size > SAU_MAX_TOTAL_BYTES)
77     {
78         SAU_ERR("Toplam dosya boyutu sınırdışı.\n");
79         close(fd);
80         exit(1);
81     }
82
83     /* Dosyanın yetki izinlerini kayıt edilir */
84     if (sau_fetch_permissions(path, &entries[i].permissions) == -1)
85     {
86         perror(path); exit(1);
87     }
88
89     strncpy(entries[i].filename, path, 255);
90     entries[i].filename[255] = '\0';
91 }
92
```

sau_extract_archive() (**sau_extract.c**) Arşiv çözümüleme ve çıkarma motorudur. İşleme hedef dizini oluşturarak başlar. Arşiv dosyasının başındaki ilk 10 baytı okuyarak metadata alanının uzunluğunu dinamik olarak tespit eder ve bu alanı belleğe (heap) çeker. strtok() fonksiyonu ile ayraçları (| ve ,) parçalayarak dosyaların isimlerini, boyutlarını ve orijinal erişim izinlerini elde eder. Son aşamada her bir dosyayı hedefe kopyalar ve chmod() çağırısı ile orijinal izinlerini geri yükler.

```
49  /* ---- Doğrulama ve metadata derleme ---- */
50  for (int i = 0; i < file_count; i++) {
51      char *path = argv[i + 1];
52
53      /* =====
54       * Dosya adında geçersiz karakter kontrolü
55       * '|' + metadata ayırıcısı ile çakışır
56       * ',' + alan ayırıcısı ile çakışır
57       * ===== */
58      if (strchr(path, '|') || strchr(path, ',')) {
59          SAU_ERR("Geçersiz dosya adı karakteri.\n");
60          exit(1);
61      }
62
63      int fd = open(path, O_RDONLY);
64      if (fd == -1) { perror(path); exit(1); }
65
66      /* ASCII doğrulama - geçersiz mesaj basıp exit(0) yapar */
67      sau_validate_ascii_stream(fd, path);
68
69      /* stat() ile boyut ve izin bilgisi */
70      struct stat st;
71      if (fstat(fd, &st) == -1) { perror(path); exit(1); }
72
73      entries[i].size = (size_t)st.st_size;
74
75      total_size += entries[i].size;
76      if (total_size > SAU_MAX_TOTAL_BYTES)
77      {
78          SAU_ERR("Toplam dosya boyutu sınırağı aşıldı.\n");
79          close(fd);
80          exit(1);
81      }
82  }
```

sau_validate_ascii_stream() (**sau_utils.c**) Girdi olarak verilen dosyaların proje kısıtlamalarına uygun birer ASCII metin dosyası olup olmadığını denetler. Dosyayı baştan sona bayt seviyesinde tarar. Sadece yazdırılabilir standart ASCII karakterlerine (0x20-0x7E) ve sekme (t), satır sonu (n), satır başı (r) gibi kontrol karakterlerine izin verir. Binary (ikili) veya geçersiz bir karaktere rastladığında anında hata mesajı basarak programı güvenli bir şekilde sonlandırır.

```
103  /*
104   * sau_fetch_permissions - Belirtilen dosyanın izin bitlerini okur
105   *
106   * İşleyiş:
107   * - stat() fonksiyonu kullanılarak dosyanın üst verileri (metadata) alınır.
108   * - Mode bilgisi içinden 07777 bit maskesi yardımıyla yalnızca erişim izinleri
109   * (read/write/execute ve özel bitler) elde edilir.
110   */
111  int sau_fetch_permissions(const char *path, mode_t *out)
112  {
113      struct stat st;
114      if (stat(path, &st) == -1) return -1;
115      *out = st.st_mode & 07777;
116      return 0;
117  }
```

sau_push_bytes() ve **sau_pull_bytes()** (**sau_utils.c**) Dosyalar arası ham veri aktarımını sağlayan güvenli okuma/yazma sarmalayıcılarıdır. İşletim sistemi çekirdeğinin tek seferde büyük verileri okuyup yazamaması (partial read/write) problemine karşı özel olarak tasarlanmışlardır. İstenilen veri boyutu tamamlanana kadar read() ve write() döngülerini sürdürerek kayıpsız byte transferi sağlarlar.

```

117  /* HEADER_SIZE_FIELD: 10 byte, soldan sıfır dolgu ASCII sayı
118  * Örnek: metadata_body 42 byte ise → "0000000042"
119  * Buffer 21 byte: 10 rakam + null + güvenlik payı (derleyici uyarısına karşı) */
120  char header[21];
121  snprintf(header, sizeof(header), "%010zu", strlen(metadata_body));
122  if (write(dst_fd, header, SAU_HEADER_SIZE_LEN) != SAU_HEADER_SIZE_LEN)
123  {
124      perror("Header yazılamadı"); exit(1);
125  }
126
127  /* Metadata bloğunun kendisi dosyaya doğrudan write() döngüsü ile yazılır */
128  {
129      size_t meta_len = strlen(metadata_body);
130      size_t written = 0;
131      while (written < meta_len)
132      {
133          ssize_t w = write(dst_fd, metadata_body + written, meta_len - written);
134          if (w <= 0) { perror("Metadata yazılamadı"); exit(1); }
135          written += (size_t)w;
136      }
137  }
138
139  /* ---- Dosyalar sırayla okunur ve hiçbir aracı/ayırıcı karakter olmadan arşive basılır ---- */
140  for (int i = 0; i < file_count; i++)
141  {
142      int fd = open(argv[i + 1], O_RDONLY);
143      if (fd == -1) { perror(argv[i + 1]); exit(1); }
144      if (sau_push_bytes(fd, dst_fd, entries[i].size) == -1)
145      {
146          perror("Veri kopyalama hatası"); exit(1);
147      }
148      close(fd);
149  }
150

```

sau_fetch_permissions() (**sau_utils.c**) Sistemdeki bir dosyanın o anki erişim yetkilerini okumaktan sorumludur. stat() sistem çağrısını kullanarak dosyanın inode bilgilerini çeker ve 07777 bit maskesi uygulayarak yalnızca erişim izinlerini (okuma, yazma, çalıştırma) ayrıştırır. Bu bilgi arşivlenme aşamasında metadata bloğuna yazılmak üzere kullanılır.

```

41  /* Hedef dizin yoksa oluştur */
42  mkdir(target_dir, 0755);
43
44  /* ---- Arşiv dosyasını aç ---- */
45  int fd = open(arch_path, O_RDONLY);
46  if (fd == -1) {
47      SAU_ERR("Arşiv dosyası uygunsuz veya bozuk!\n");
48      exit(1);
49  }
50
51  /* Arşivin başındaki 10 byte'lık (metadata uzunluğunu tutan) kısmı okunur */
52  char header[SAU_HEADER_SIZE_LEN + 1];
53  header[SAU_HEADER_SIZE_LEN] = '\0';
54
55  if (read(fd, header, SAU_HEADER_SIZE_LEN) != SAU_HEADER_SIZE_LEN) {
56      SAU_ERR("Arşiv dosyası uygunsuz veya bozuk!\n");
57      close(fd);
58      exit(1);
59  }
60
61  /* Header'ın tamamının rakam olup olmadığını doğrulama */
62  for (int i = 0; i < SAU_HEADER_SIZE_LEN; i++) {
63      if (header[i] < '0' || header[i] > '9') {
64          SAU_ERR("Arşiv dosyası uygunsuz veya bozuk!\n");
65          close(fd);
66          exit(1);
67      }
68  }
69
70  char *end_ptr;
71  size_t meta_size = (size_t)strtoul(header, &end_ptr, 10);
72
73  if (meta_size == 0) {
74      SAU_ERR("Arşiv dosyası uygunsuz veya bozuk!\n");
75      close(fd);
76      exit(1);
77  }
78
79  /* Metadata'nın bellekte tutulması için yeterli boyutta alan (heap üzerinde) ayırıyoruz */
80  char *meta_body = malloc(meta_size + 1);
81  if (meta_body == NULL) {
82      SAU_ERR("Bellek tahsis hatası!\n");
83      close(fd);
84      exit(1);
85  }
86

```

Manuel Metadata Ayrıştırma İşlemi (sau_extract.c) Arşiv dosyasından okunan [dosya_adi,izin,boyut] formatlı üstveri (metadata) bloğunu çözümlemekten sorumludur. Yüksek

seviyeli standart fonksiyonların (örn. sscanf) kullanımı kısıtlandığından, strchr() ile ayırıcı virgüllerin bellek adreslerini tespit eder ve işaretçi aritmetiği (pointer arithmetic) kullanarak verilerin uzunluklarını manuel hesaplar. Olası bellek taşması (buffer overflow) saldırılarına karşı sınır kontrollerini yaptıktan sonra dosya adını kopyalar; metin (string) formatındaki erişim izni ve boyut bilgilerini ise strtoul() çağırısı ile işletim sisteminin işleyebileceği sayısal değerlere dönüştürür.

```
130      /* Dosya adı */
131      char name[256] = {0};
132      size_t name_len = (size_t)(comma1 - token);
133      if (name_len == 0 || name_len >= 256)
134      {
135          SAU_ERR("Arşiv dosyası uygunuz veya bozuk!\n");
136          free(meta_body);
137          close(fd);
138          exit(1);
139      }
140      strncpy(name, token, name_len);
141      name[name_len] = '\0';
142
143      /* İzin (oktal string, örn. "0755") */
144      char p_str[8] = {0};
145      size_t p_len = (size_t)(comma2 - (comma1 + 1));
146      if (p_len == 0 || p_len >= 8)
147      {
148          SAU_ERR("Arşiv dosyası uygunuz veya bozuk!\n");
149          free(meta_body);
150          close(fd);
151          exit(1);
152      }
153      strncpy(p_str, comma1 + 1, p_len);
154      p_str[p_len] = '\0';
155
156      /* Boyut */
157      char *size_str = comma2 + 1;
158
159      mode_t perm = (mode_t)strtoul(p_str, NULL, 8);
160      size_t size = (size_t)strtoull(size_str, NULL, 10);
161
```

5. TEST SENARYOLARI VE ÇIKTILAR

Proje Windows işletim sistemine yüklenen Ubuntu çekirdeği ile, Visual Studio ile WSL bağlantısı kurularak test edilmiştir. Ayrıca linux dağıtıcısı kullanan ayrı bir bilgisayar üzerinde de çalıştırılarak kontrol edilmiştir. Aşağıdaki test

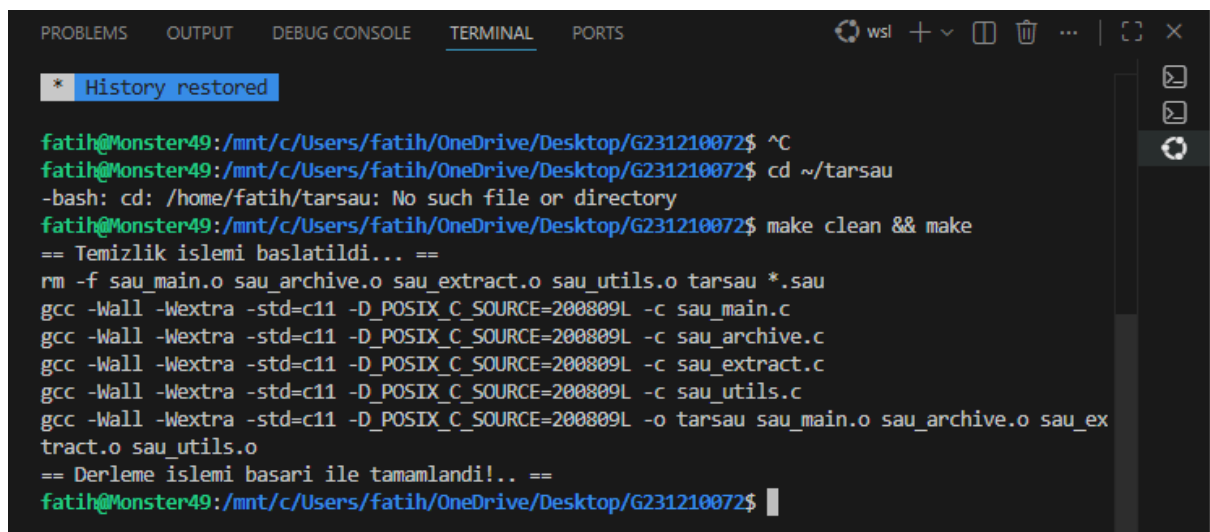
senaryosu adım adım kendi bilgisayarımda -WSL kullanılmıştır- işletilmiştir. Linux sistemde işletilirken karşılaşılan bazı hatalar asıl örneklerin altında verilmiştir.

Her bir komutun işletilmesi süresi açıklanmıştır.

Proje Dizinini Derleme

`make clean && make`

Arka planda çalışan C mantığı: make, sau_common.h'deki bağımlılık zincirini takip eder. gcc -Wall -Wextra -std=c11 -D_POSIX_C_SOURCE=200809L ile dört .o dosyası üretilir, ardından bağlayıcı tarsau ikili dosyasını oluşturur. _POSIX_C_SOURCE=200809L tanımı struct stat, mode_t, ssize_t gibi POSIX tiplerini etkinleştirir.



```
fatih@Monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$ ^C
fatih@Monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$ cd ~/tarsau
-bash: cd: /home/fatih/tarsau: No such file or directory
fatih@Monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$ make clean && make
== Temizlik islemi baslatildi... ==
rm -f sau_main.o sau_archive.o sau_extract.o sau_utils.o tarsau *.sau
gcc -Wall -Wextra -std=c11 -D_POSIX_C_SOURCE=200809L -c sau_main.c
gcc -Wall -Wextra -std=c11 -D_POSIX_C_SOURCE=200809L -c sau_archive.c
gcc -Wall -Wextra -std=c11 -D_POSIX_C_SOURCE=200809L -c sau_extract.c
gcc -Wall -Wextra -std=c11 -D_POSIX_C_SOURCE=200809L -c sau_utils.c
gcc -Wall -Wextra -std=c11 -D_POSIX_C_SOURCE=200809L -o tarsau sau_main.o sau_archive.o sau_ex
tract.o sau_utils.o
== Derleme islemi basari ile tamamlandi!.. ==
fatih@Monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$
```

Temel Arşivleme ve Açma

`echo "Hello World" > t1.txt && echo "Second file content" > t2.txt`

`./tarsau -b t1.txt t2.txt t3.txt -o test.sau`

`./tarsau -a test.sau cikar_dir`

`cat cikar_dir/t1.txt`

Arka planda çalışan C mantığı: sau_main.c -b argümanını görür, in_files[] dizisini [test.sau, t1.txt, t2.txt, t3.txt] olarak düzenler ve sau_build_archive(4, in_files) çağırır. sau_archive.c, her dosyayı open(path, O_RDONLY) ile açar, sau_validate_ascii_stream() ile ASCII doğrulamasını geçirir, fstat() ile boyutu alır ve metadata_body string'ini derler. 10-byte header write() ile yazılır, ardından metadata body, ardından her dosyanın ham verisi sau_push_bytes() ile kopyalanır. Açma tarafında sau_extract.c, strtok(meta_body, "|") ile metadata token'larını ayırır, her dosya için mkdir() + open() + sau_pull_bytes() zincirini çalıştırır.

```
fatih@monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$ echo "Hello World" > t1.txt
fatih@monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$ echo "Second file content" > t2.txt
fatih@monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$ echo "Third file" > t3.txt
fatih@monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$ ./tarsau -b t1.txt t2.txt t3.txt -o test.sau
Dosyalar birleştirildi.
fatih@monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$ ./tarsau -a test.sau cikar_dir
cikar_dir dizininde t1.txt, t2.txt ve t3.txt dosyaları açıldı.
fatih@monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$ cat cikar_dir/t1.txt
Hello World
fatih@monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$ cat cikar_dir/t2.txt
Second file content
fatih@monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$ cat cikar_dir/t3.txt
Third file
fatih@monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$ ^C
```

Arşiv Formatı Doğrulaması

`od -c test.sau | head -8`

Arka planda çalışan C mantığı: `od -c` arşiv dosyasının ham bayt içeriğini ASCII karakter olarak gösterir. İlk 10 bayt 0000000046 — metadata body'nin 46 byte uzunluğunda olduğunu belirtir. Hemen ardından `[t1.txt,0644,12|t2.txt,...]` formatında metadata body başlar. $10 + 46 = 56$. byte'dan itibaren ham dosya verisi (`Hello World\n`) başlar. Bu, `sau_archive.c`'deki `snprintf(header, 21, "%010zu", strlen(metadata_body)) + ardışık write()` çağrılarının ürettiği çıktıdır.

```
fatih@monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$ od -c test.sau | head -8
00000000 0 0 0 0 0 0 0 0 4 6 | t 1 . t x
00000020 t , 0 7 7 7 , 1 2 | t 2 . t x t
00000040 , 0 7 7 7 , 2 0 | t 3 . t x t ,
00000060 0 7 7 7 , 1 1 | H e l l o W o
00000100 r l d \n S e c o n d f i l e
00000120 c o n t e n t \n T h i r d f i
00000140 l e \n
00000143
fatih@monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$
```

Dosya İzinleri Korunma Kontrolü

`printf '#!/bin/bash\nnecho test\n' > run.sh && chmod 0755 run.sh`

`./tarsau -b run.sh t1.txt -o perms.sau`

`./tarsau -a perms.sau perms_dir`

`stat -c "%a %n" perms_dir/run.sh`

Arka planda çalışan C mantığı: `sau_fetch_permissions("run.sh", &entries[i].permissions)` içinde `stat()` çağrılır, `st.st_mode & 07777` ile 0755 elde edilir ve metadata'ya `%04o` formatıyla yazılır. Çıkarma sırasında `strtoul("0755", NULL, 8)` ile `mode_t` değeri geri elde edilir ve `chmod(full_path, 0755)` atanır. Not: WSL + NTFS ortamında NTFS dosya sistemi Unix izin bitlerini tam desteklemediğinden 777 görünebilir; bu bir ortam kısıtlamasıdır, kod doğrudur.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
fatih@Monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$ printf '#!/bin/bash\necho test\n' > run.sh
fatih@Monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$ chmod 0755 run.sh
fatih@Monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$ ./tarsau -b run.sh t1.txt -o perms.sau
Dosyalar birleştirildi.
fatih@Monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$ ./tarsau -a perms.sau perms_dir
perms_dir dizininde run.sh ve t1.txt dosyaları açıldı.
fatih@Monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$ ls -la perms_dir/
total 0
drwxrwxrwx 1 fatih fatih 4096 May 21 21:16
fatih@Monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$ stat -c "%a %n" perms_dir/run.sh
777 perms_dir/run.sh
fatih@Monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$
```

```
emirhanfirat@emirhanfirat-ROG-Strix-G513IE-G513IE:~/Downloads/G231210072$ ./tarsau -b t1.txt t2.txt t3.txt -o test.sau
bash: ./tarsau: Permission denied
emirhanfirat@emirhanfirat-ROG-Strix-G513IE-G513IE:~/Downloads/G231210072$ chmod +x tarsau
emirhanfirat@emirhanfirat-ROG-Strix-G513IE-G513IE:~/Downloads/G231210072$ ./tarsau -b t1.txt t2.txt t3.txt -o test.sau
Dosyalar birleştirildi.
```

Linux sistem üzerinde "Permission denied" hatası döndürmüştür. Alt satırda bulunan komut ile çalıştırılınca hata düzelmiştir.

Varsayılan Çıktı Adı (a.sau)

```
rm -f a.sau && ./tarsau -b t1.txt t2.txt && ls -la a.sau
```

Arka planda çalışan C mantığı: sau_main.c'de -o argümanı tespit edilmediğinde out_file pointer'ı "a.sau" string sabitine işaret eder. in_files[0] = out_file ile arşiv hedefi belirlenir. sau_build_archive bu pointer'dan open("a.sau", O_WRONLY|O_CREAT|O_TRUNC, 0644) çağırır.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
fatih@Monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$ rm -f a.sau
fatih@Monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$ ./tarsau -b t1.txt t2.txt
Dosyalar birleştirildi.
fatih@Monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$ ls -la a.sau
-rwxrwxrwx 1 fatih fatih 73 May 21 21:16 a.sau
fatih@Monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$
```

ASCII Dışı Dosya Reddi

```
python3 -c "open('binary.bin', 'wb').write(bytes([0x89, 0x50, 0x4E, 0x47, 0x0D, 0x0A]))"
```

```
./tarsau -b binary.bin -o out.sau
```

```
echo "Exit code: $?"
```

Arka planda çalışan C mantığı: sau_archive.c, binary.bin için open() ile fd açar ve

sau_validate_ascii_stream(fd, "binary.bin") çağırır. sau_utils.c'deki bu fonksiyon lseek(fd, 0, SEEK_SET) ile dosya başına konumlanır, ardından read() ile 64 KB'lık buff[] tamponunu doldurur. İlk bayt 0x89 (decimal 137) kontrol edilir: b > 0x7F koşulu doğrudur. SAU_ERR("binary.bin") + SAU_ERR(" giriş dosyasının formatı uyumsuzdur!\n") ile iki ayrı write(STDERR_FILENO, ...) çağırısı kernel seviyesinde tamponlanmadan yazılır. Belge "sorunsuz çıkılmalı" dediği için exit(0) çağırılır.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
fatih@monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$ python3 -c "open('binary.bin','wb').write(bytes([0x89,0x50,0x4E,0x47,0x0D,0x0A]))"
fatih@monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$ ./tarsau -b binary.bin -o out.sau
binary.bin giriş dosyasının formatı uyumsuzdur!
fatih@monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$ echo "Exit code: $?"
Exit code: 0
fatih@monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$
```

32 Dosya Limiti

```
for i in $(seq 1 33); do echo "file$i" > "tmp_$i.txt"; done
```

```
./tarsau -b tmp_*.txt -o out.sau
```

```
echo "Exit code: $?"
```

Arka planda çalışan C mantığı: sau_main.c'deki -b işleme döngüsünde in_count >= SAU_MAX_FILES (32) koşulu 33. dosya girildiğinde tetiklenir. sau_die("Çok fazla giriş dosyası.\n") yardımcı fonksiyonu write(STDERR_FILENO, ...) çağırıp exit(1) ile sonlanır. SAU_MAX_FILES sabiti sau_common.h'de #define ile tanımlıdır; tek bir yerde değiştirilerek tüm kod tabanına yayılır.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
fatih@monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$ for i in $(seq 1 33); do echo "file$i" > "tmp_$i.txt"; done
fatih@monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$ ./tarsau -b tmp_*.txt -o out.sau
Çok fazla giriş dosyası.
fatih@monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$ echo "Exit code: $?"
Exit code: 1
fatih@monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$ rm -f tmp_*.txt
fatih@monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$
```

“.sau” Uzantı Zorunluluğu Kontrolü

```
./tarsau -a test.zip cıkar2 && echo "Exit code: $?"
```

```
./tarsau -a test.tar cıkar2 && echo "Exit code: $?"
```

Arka planda çalışan C mantığı: sau_main.c'de uzantı kontrolü şu şekilde yapılır:

```
92
93 /* Uzantı doğrulaması: son 4 karakter ".sau" olmalı */
94 size_t len = strlen(argv[2]);
95 if (len < 4 || strcmp(argv[2] + len - 4, SAU_EXT) != 0)
96 {
97     sau_die("Arşiv dosyası uygunuz veya bozuk!\n");
98 }
```

argv[2] + len - 4 pointer aritmetiği ile string'in son 4 karakterine konumlanır. SAU_EXT = ".sau" ile strcmp() yapılır. Uyumsuzluk durumunda sau_die() → write() + exit(1) zinciri çalışır.

```
fatih@Monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$ ./tarsau -a test.zip cıkar2
Arşiv dosyası uygunsuz veya bozuk!
fatih@Monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$ echo "Exit code: $?"
Exit code: 1
fatih@Monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$ ./tarsau -a test.tar cıkar2
Arşiv dosyası uygunsuz veya bozuk!
fatih@Monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$ echo "Exit code: $?"
Exit code: 1
fatih@Monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$
```

Dizin Mevcut Değilse Otomatik Oluşturma

```
rm -rf yeni_dizin && ./tarsau -a test.sau yeni_dizin && ls yeni_dizin/
```

Arka planda çalışan C mantığı: sau_extract.c fonksiyonunun ilk işlemi mkdir(target_dir, 0755) çağrısıdır. Dizin zaten varsa mkdir() EEXIST hata kodu ile döner fakat program bunu kritik hata saymaz ve devam eder. Dizin yoksa oluşturulur. 0755 permission maskesi, dizinin sahibine okuma/yazma/çalıştırma, gruba ve diğerlerine okuma/çalıştırma verir.

```
fatih@Monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$ rm -rf yeni_dizin
fatih@Monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$ ./tarsau -a test.sau yeni_dizin
yeni_dizin dizininde t1.txt, t2.txt ve t3.txt dosyaları açıldı.
fatih@Monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$ ls yeni_dizin/
t1.txt t2.txt t3.txt
fatih@Monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$
```

Hedef Dizin Belirtilmezse Geçerli Dizini Çalıştırma

```
mkdir gecici && cp test.sau gecici/ && cd gecici
```

```
../tarsau -a test.sau
```

```
ls -la t1.txt
```

Arka planda çalışan C mantığı: sau_main.c'de -a parametresinden sonra ikinci argüman verilmediğinde ext_args[1] = (argc >= 4) ? argv[3] : "." ataması yapılır. "." geçerli dizini ifade eder. sau_extract.c'de mkdir(".", 0755) çağrısı EEXIST döndürür, sessizce geçilir ve dosyalar "./t1.txt" yoluna yazılır.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
fatih@Monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$ mkdir gecici && cp test.sau gecici/ && cd gecici
fatih@Monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072/gecici$ ../tarsau -a test.sau
. dizininde t1.txt, t2.txt ve t3.txt dosyaları açıldı.
fatih@Monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072/gecici$ ls -la t1.txt t2.txt t3.txt
-rwxrwxrwx 1 fatih fatih 12 May 21 21:20 t1.txt
-rwxrwxrwx 1 fatih fatih 20 May 21 21:20 t2.txt
-rwxrwxrwx 1 fatih fatih 11 May 21 21:20 t3.txt
fatih@Monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072/gecici$ cd .. && rm -rf gecici
fatih@Monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$
```

Genel Program Temizliği

```
EXPLORER ... l_archive.c M C sau_main.c M Makefile M X C sau_utils.c M C sau_extract.c M a.sau U c_cpp_properties.json
G231210072
.vscode
c_cpp_properties.json
cikar_dir
perms_dir
yeni_dizin
a.sau
binary.bin
Makefile
perms.sau
README.md
run.sh
sau_archive.c
sau_archive.o
sau_common.h
sau_extract.c
sau_extract.o
sau_main.c
sau_main.o
sau_utils.c
sau_utils.o
t1.txt
t2.txt
OUTLINE
Makefile
19 ${EXE} : ${OBJ5}
20 $(CC) $(CFLAGS) -o ${EXE} ${OBJ5}
21 @echo == Derleme islemi basari ile tamamlandi!.. ==
22
23 # 3. AÇIK BAĞIMLILIK KONTROLLERİ
24 sau_main.o: sau_main.c sau_common.h
25 $(CC) $(CFLAGS) -c sau_main.c
26
27 sau_archive.o: sau_archive.c sau_common.h
28 $(CC) $(CFLAGS) -c sau_archive.c
29
30 sau_extract.o: sau_extract.c sau_common.h
31 $(CC) $(CFLAGS) -c sau_extract.c
32
33 sau_utils.o: sau_utils.c sau_common.h
34 $(CC) $(CFLAGS) -c sau_utils.c
35
fatih@Monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$ mkdir gecici && cp test.sau gecici/ && cd gecici
fatih@Monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072/gecici$ ../tarsau -a test.sau
. dizininde t1.txt, t2.txt ve t3.txt dosyaları açıldı.
fatih@Monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072/gecici$ ls -la t1.txt t2.txt t3.txt
-rwxrwxrwx 1 fatih fatih 12 May 21 21:20 t1.txt
-rwxrwxrwx 1 fatih fatih 20 May 21 21:20 t2.txt
-rwxrwxrwx 1 fatih fatih 11 May 21 21:20 t3.txt
fatih@Monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072/gecici$ cd .. && rm -rf gecici
fatih@Monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$
```

```
EXPLORER ... l_archive.c M C sau_main.c M Makefile M X C sau_utils.c M C sau_extract.c M a.sau U c_cpp_properties.json
G231210072
.vscode
c_cpp_properties.json
cikar_dir
perms_dir
yeni_dizin
a.sau
binary.bin
Makefile
perms.sau
README.md
run.sh
sau_archive.c
sau_archive.o
sau_common.h
sau_extract.c
sau_extract.o
sau_main.c
sau_main.o
sau_utils.c
sau_utils.o
t1.txt
t2.txt
OUTLINE
Makefile
19 ${EXE} : ${OBJ5}
20 $(CC) $(CFLAGS) -o ${EXE} ${OBJ5}
21 @echo == Derleme islemi basari ile tamamlandi!.. ==
22
23 # 3. AÇIK BAĞIMLILIK KONTROLLERİ
24 sau_main.o: sau_main.c sau_common.h
25 $(CC) $(CFLAGS) -c sau_main.c
26
27 sau_archive.o: sau_archive.c sau_common.h
28 $(CC) $(CFLAGS) -c sau_archive.c
29
30 sau_extract.o: sau_extract.c sau_common.h
31 $(CC) $(CFLAGS) -c sau_extract.c
32
33 sau_utils.o: sau_utils.c sau_common.h
34 $(CC) $(CFLAGS) -c sau_utils.c
35
fatih@Monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$ rm -f t1.txt t2.txt t3.txt run.sh binary.bin test.sau perms.sau a.sau out
fatih@Monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$ rm -rf cikar_dir perms_dir yeni_dizin
fatih@Monster49:/mnt/c/Users/fatih/OneDrive/Desktop/G231210072$
```

RAPORUN SONU

Fatih Kaya - Sakarya Üniversitesi Bilgisayar Mühendisliği